SCALER School of Technology · Priyansh Saxena

ML System Design Rubric

Part 1 of 2 ▪ The 10-Step Framework

Priyansh Saxena

Scaler School of Technology | ML System Design and LLMOps

Week 5 — Session 1/2 (Part 1) /

Why a Rubric?

What interviewers actually assess

- Structure — do you cover all the right dimensions?
- Completeness — do you miss obvious concerns?
- Trade-off reasoning — can you justify your choices?

What they do *not* expect

- The most complex model
- Perfect numbers (rough estimates are fine)
- Knowledge of every tool by name

The rubric in 10 steps

1. Clarify the Problem
2. Context
3. Problem Statement
4. Define Requirements
5. Estimations
6. System Architecture
7. Evaluation
8. A/B Testing
9. Deployment & Monitoring
10. Risk & Trade-offs

1 Clarify the Problem

Before designing anything, ask questions.

- What must the system achieve?
- What metrics define success?
- What are the constraints? (time, data, legal)
- What is out of scope?

Why this matters

The problem brief is *deliberately incomplete*. Asking good questions shows you think like a product engineer, not just a model builder.

Common clarifying questions

- How many users / items / events?
- What latency SLA is acceptable?
- Real-time or batch?
- Is data available and labelled?
- Privacy / regulatory constraints?
- Cost of a bad prediction?

2 Context

Situate the system in the real world.

- What business problem does this solve?
- Who are the users and what do they need?
- What happens if the system fails?
- What existing systems does this replace?

Key principle

Context shapes every decision — model choice, latency budget, fairness requirements, and what counts as success.

Example

A news recommender and an e-commerce checkout recommender use the same algorithms; context determines which trade-offs matter.

3 Problem Statement

Formally define the ML problem.

- What is the **input**?
- What is the **output**?
- What does the model **predict, rank, or generate**?
- How does the ML output map to a **business action**?

Good vs. weak

Weak: "Build a recommendation system."

Good: "Given a user's interaction history, rank 1,000 candidate songs by probability of play and return the top 20 to the app home screen."

Precision here saves time later

A vague problem statement leads to architecture debates 30 minutes in. Nail it here.

4 Define Requirements

Functional — what it must *do*

- Core capabilities and features
- Expected outputs
- User-facing behaviours

e.g. personalised ranking, cold-start, 24h song freshness

Non-functional — how it must *behave*

- Latency SLA (p99, not mean)
- Throughput (RPS)
- Availability (99.9% = 8.7h downtime/yr)
- Scalability (10× headroom)
- Privacy / compliance (GDPR)
- Maintainability

Non-functional requirements drive architecture

A 100ms SLA forces a two-stage design. A 10s SLA allows a single ranker. Requirements are architectural constraints.

5 Estimations

Size the system before designing it.

- Users / items / events per day
- Request volume (QPS, peak vs. average)
- Data volume per day
- Storage (features, models, logs)
- Compute (training hours, serving pods)

Why estimate?

"Rank 100M songs in real-time" fails the estimation test immediately — no need to build it to know it's wrong.

Back-of-envelope template

What	Formula
QPS	DAU $\times$ sessions / 86400
Storage/day	events $\times$ bytes/event
Feature memory	items $\times$ dims $\times$ 4B
Serving pods	QPS $\times$ latency

Off by 2$\times$ is fine. Off by 1000$\times$ is a design error.

6 System Architecture



(a) sources, signals, ingestion pattern
(b) ETL, storage, schema, quality
(c) feature store (offline + online)
(d) candidate generation, data prep
(e) algorithm, training pipeline, registry
(f) re-ranking, filtering, business rules

Design principle

For every box: *What does it do? How do you use it? What does it depend on?* If you can't answer all three, the boundary needs work.

7 Evaluation: Offline and Online

Offline (before deployment)

- **Precision@K / Recall@K** — relevant items in top-K?
- **NDCG@K** — relevant items ranked highest?
- **AUC-ROC** — click prediction quality

Fast and cheap. Does not measure real behaviour.

Online (after deployment)

- **North star:** CTR, session length
- **Guardrails:** diversity, error rate, latency
- **Good-to-have:** discovery rate, retention

Ground truth. Slow and expensive to collect.

The gap between offline and online

High NDCG $\not\Rightarrow$ high CTR. Offline measures *predicted* relevance; online measures *actual* behaviour. The gap is where interesting problems live.

8 A/B Testing

What to define

- **Unit:** what gets randomised? (user / session)
- **Groups:** treatment (new) vs. control (current)
- **Split:** gradual ramp — 10% → 50% → 100%
- **Duration:** ≥2 weeks to capture weekly patterns
- **Sample size:** power analysis; smaller effect = larger sample

Biases to flag

- **Selection bias** — unbalanced groups
- **Novelty effect** — users engage with anything new
- **Cannibalism** — treatment affects control
- **Simpson's paradox** — aggregate reversal

Decision rule

Promote if primary metric improves **and** no guardrail degrades. Rollback if latency or error rate exceeds threshold within 24h.

9 Deployment and Monitoring

Deployment

- **Pipelines:** Airflow / Kubeflow DAGs
- **Rollout:** canary (5%→100%) or blue-green
- **Registry:** Staging → Production → Archived
- **Serving API:** REST/gRPC; Triton / Vertex AI

Monitoring — 4 layers

1. **Data quality** — schema, nulls, volume
2. **Feature drift** — vs. training baseline
3. **Prediction drift** — output distribution
4. **Business metrics** — CTR, revenue

Alert ordering matters

Data quality → feature drift → prediction drift → business impact. Don't wait for the last one to act.

10 Risk Management and Trade-offs

Technical trade-offs

- Accuracy vs. latency (bigger model = slower)
- Freshness vs. cost (real-time = expensive)
- Personalisation vs. cold-start
- Diversity vs. CTR
- Complexity vs. maintainability

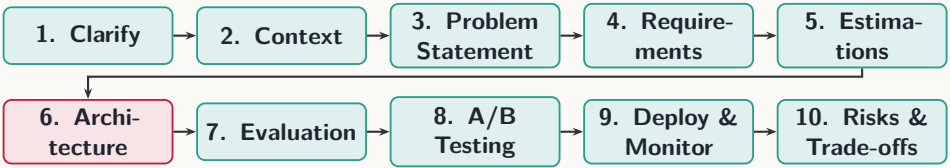
Ethical and fairness risks

- **Popularity bias:** rich-get-richer
- **Filter bubbles:** users see only what they like
- **Feedback loops:** outputs become training data
- **Demographic bias:** protected attributes leak

Interview tip

Proactively raise trade-offs: "I chose X over Y because Z. The cost of this choice is . . . " Senior candidates don't wait to be asked.

The 10-Step Rubric at a Glance



Steps 1–5: Frame the problem.
Most candidates rush these. Don't.

Step 6: Design the system. Most time spent here.
Steps 7–10: Where senior candidates stand out.

Up next: the Case Study

Part 2 applies all ten rubric steps to a real-time music recommendation system.



Priyansh Saxena

https://www.priyanshsaxena.com/sst-ml-system-design